Cifar-10 image classifier

Selected models from training history

Model name	T. Accuracy	Val Loss	Epochs	Learning Rate	Augmentation Layers	Callbacks	Optimizer
cifar10_20251028_03	87,57%	0,39	100	1,00E-03	Flip, Rotation, Zoom, Erasing	ED, RLRP	Adam
cifar10_TF_ResNet50	93,09%	0,21	3 / 50	1e-3 / 1e-5	Flip, Rotation, Zoom	ED, RLRP	Adam

- Time Allocation: Fine-tuning and iterating 70%, Data Processing and Normalization
- Stochastic Gradient Descent (SGD), Adam and AdamW optimizers were tested, but only best results are highlighted in the table.
- The number of epochs and learning rate was adjusted based on the observed accuracy in previous iterations on the fly

Model architecture overview

Custom Model	Transfer learning: ResNet50
- Data Augmentation: Applied directly within the model to enhance training data diversity. - Convolutional Layers: 3 hidden layers that combine two Conv2D layers using an increasing amount of filters, followed by BatchNormalization for improved model convergence, MaxPooling and a Dropout to prevent overfitting. - The last layer of the architecture consists of flattening the data, a Dense layer with BatchNormalization.	- Pre-trained on ImageNet, the top classification layer removed, as the original model has 1000 output nodes for its 1000 classes, for Cifar-10 we only need 10 (0, 9). - Classification Head: GlobalAveragePooling2D, followed by a Dense layer with BatchNormalization, Dropout, and a Dense output layer. Similar to the custom model. - First the model was trained using ResNet50 for an initial phase of three epochs with a higher learning rate, once completed, the model weights were saved. For fine-tuning, these weights were restored, then the model was trained for roughly 50 epochs to finely tune its performance.

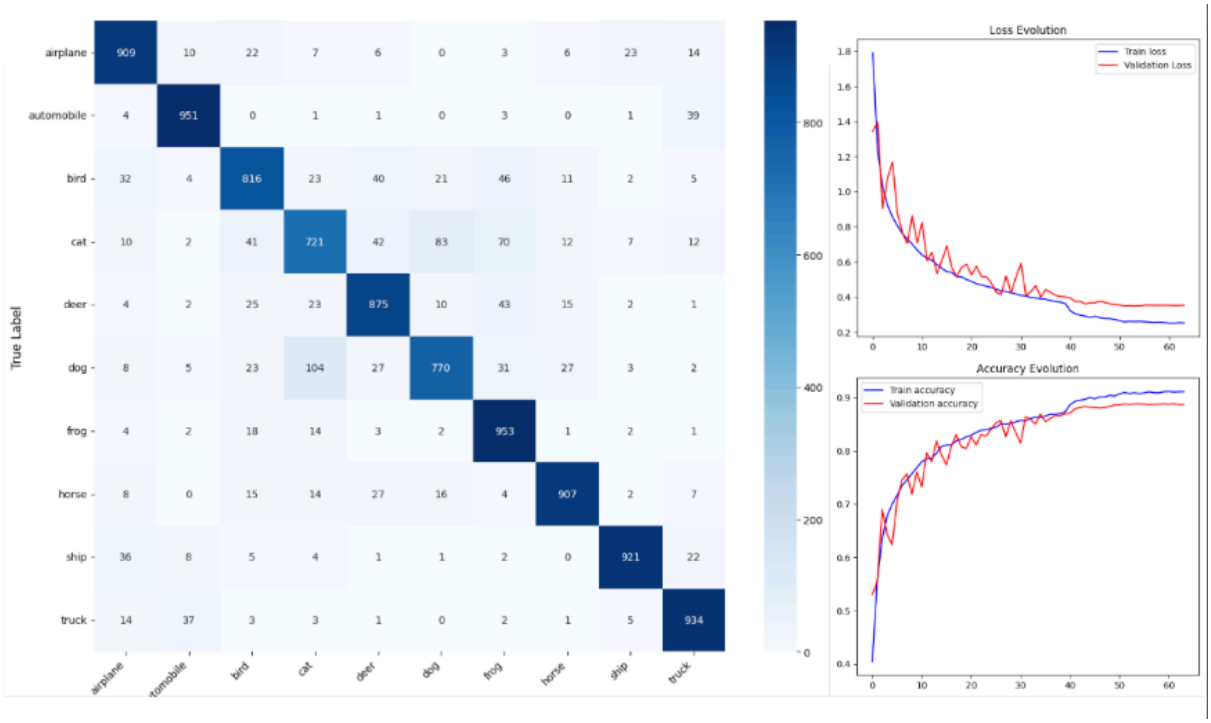
Conclusions and key takeaways

CIFAR-10 Dataset Insights: The CIFAR-10 dataset, commonly recognized as "challenging," provided both difficulties and advantages. While it posed a high level of difficulty, its uniform class image count and standardized size of 32x32 pixels greatly facilitated the preprocessing and normalization of pixel values, making model training and evaluation more streamlined.

Deployment Considerations: As previously noted, deploying a 15 MB custom model significantly eased the process of creating, and pushing images to Google Cloud Platform (free tier :)). This smaller model size also enhances user experience, with an image startup time of around 10 seconds. Conversely, deploying a larger, 250 MB model can be slow and costly due to the need for continuous service readiness, ultimately inflating operational expenses. This trade-off for a mere 5 or 6 points increase in accuracy may not be justified. But also, it depends on actual and desired use of a solution, for example for a medical or healthcare where accuracy is paramount it may be justified.

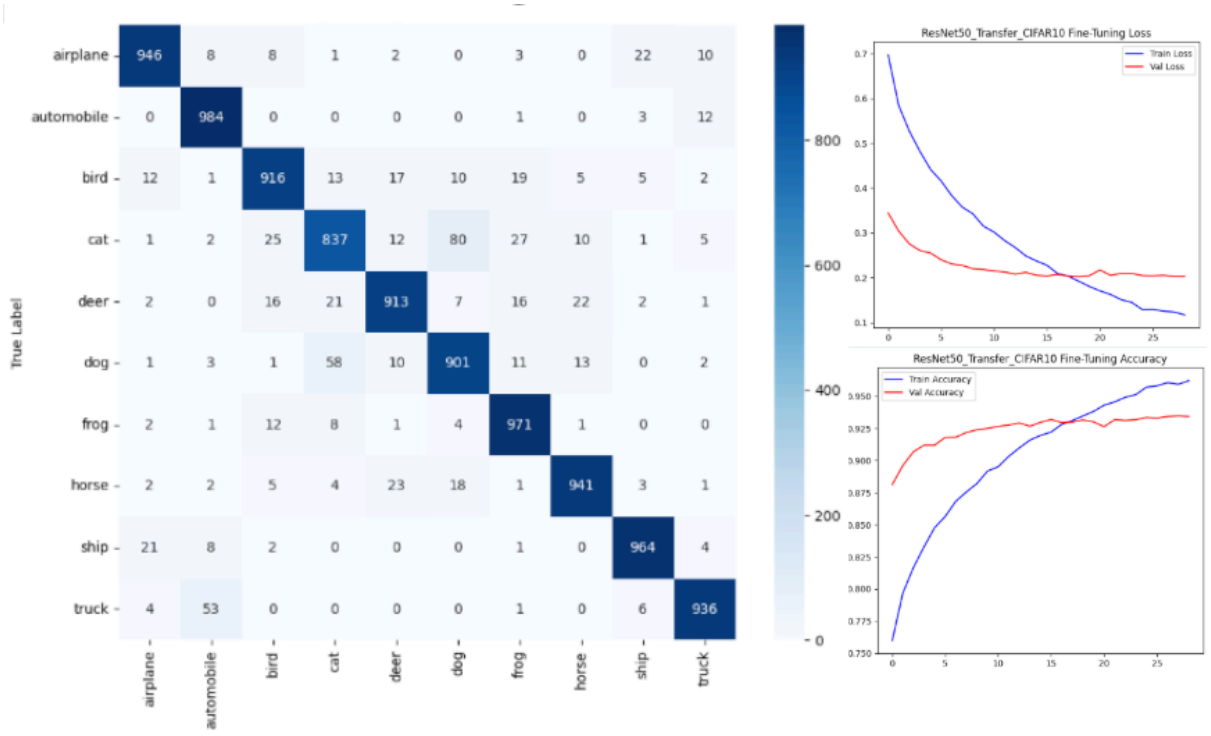
Welcome to the realm of transfer learning: Significant effort and retries were necessary due to the initial attempt to create a single notebook that was able to control and switch between several custom and pre-trained models in a configurable way. What became apparent, however, and I did know it beforehand, is that each pre-trained model often requires specific input configurations. For example, some models expect images to have their color channels adjusted from RGB to BGR, like ResNet and VGG architectures. Additionally, standard normalization to a 0-1 range doesn't suit all models; some, like those based on Inception, require specific pre-processing linked to their original training datasets. To manage these variations effectively, I found it most organized to create a dedicated notebook for each type of transfer learning model, allowing tailored input preparation and model configuration.

Custom model results



The custom model achieved a macro average precision and recall of 0.88, with a weighted average precision and recall also at 0.88, and an f1-score of 0.87."

Transfer learning ResNet50 model result



The transfer learning model demonstrated significantly higher accuracy from the start, achieving around 86% validation accuracy by the first epoch. However, this precision comes with greater time and computational demands. Themodel achieved an overall **accuracy of 93%** on the test set of 10,000 samples, with both macro and weighted averages showing **precision, recall, and f1-score metrics consistently at 0.93.**